Performance Measurement(Search)

王文浩

Date: 2026-3-20

Chapter 1：引言

本章简要介绍问题描述，任务目标

1、问题描述

给定一个包含 N 个有序整数的列表（编号 0 到 N-1），查找其中的某个值的不同算法，并分析不同方法在最坏情况的时间对比

2、算法背景

- 顺序搜索：通过遍历整个数组和目标值对比来判断目标值是否出现于列表中，最简单的算法，无论列表是否有序都可以使用。
- 二分查找：使用分治策略，在有序列表中，每次对比可排除其中一半数据肯定不符合要求，从而减小数据规模，获得更好的性能，但是前提是列表有序

3、任务目标

- 算法：分别用**迭代和递归**的方法实现**顺序搜索(Sequential Search)** 和 **二分搜索(Binary Search)** 这两种基础算法
- 最坏情况: 分析最坏情况，即让算法耗时最长的特殊情况。
- 比较运行时间：对于不同规模的数据，测量并比较四种算法再最快情况下的实际使用时间。

4、问题分析

- Worst case: 对于一个有序列表，查找一个不在列表中的元素 会使算法进行最大次数的比较，耗时最长。

Chapter 2：算法规范

1、解题思路

1. **算法描述**
    i. **迭代二分查找(Iterative Binary Search)**

- 核心思路：每次比较中间元素，根据大小缩小搜索区间
- 伪代码：

```
输入: 有序数组 a[0..N-1], 数组长度 N, 目标值 key
输出: 如果找到返回 1, 否则返回 0

low = 0
high = N - 1
while low <= high do
    mid = (low + high) / 2
    if a[mid] == key then
        return 1
    else if a[mid] < key then
        low = mid + 1
    else
        high = mid - 1
    end if
end while
return 0
```

- 实际代码：

```c
int Iterative_Binary_Search(int a[], int N, int key) {
    int low = 0, high = N - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key) {
            return 1;
        }
        else if (a[mid] < key) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }
    return 0;
}
```

ii. 递归二分查找(Rescursive Binary Search)

- 核心思路，同迭代二分查找，但通过递归调用缩小区间
- 伪代码：

```
输入: 有序数组 a[0..N-1], 搜索区间下界 low, 上界 high, 目标值 key
输出: 如果找到返回 1, 否则返回 0

if low > high then
    return 0
end if
mid = (low + high) / 2
if a[mid] == key then
    return 1
else if a[mid] < key then
    return Recursive_Binary_Search(a, mid + 1, high, key)
else
    return Recursive_Binary_Search(a, low, mid - 1, key)
end if
```

- 实际代码：

```c
int Recursive_Binary_Search(int a[], int low, int high, int key) {
    if (low > high) {
        return 0;
    }
    int mid = (low + high) / 2;
    if (a[mid] == key) {
        return 1;
    }
    else if (a[mid] < key) {
        return Recursive_Binary_Search(a, mid + 1, high, key);
    }
    else {
        return Recursive_Binary_Search(a, low, mid - 1, key);
    }
}
```

iii. **迭代顺序查找(Iterative Sequential Search)**

- 核心思路：从第一个元素开始依次比较，直到找到目标，或者遍历完数组。
- 伪代码：

```
输入: 数组 a[0..N-1], 数组长度 N, 目标值 key
输出: 如果找到返回 1, 否则返回 0

for i = 0 to N-1 do
    if a[i] == key then
        return 1
    end if
end for
return 0
```

- 实际代码：

```c
int Iterative_Sequential_Search(int a[], int N, int key) {
    for (int i = 0; i < N; i++) {
        if (a[i] == key) {
            return 1;
        }
    }
    return 0;
}
```

---

iv. **递归顺序查找(Recursive Sequential Search)**

- 核心思路：每次检查最后一个元素是否和key一致，递归处理剩余部分，直到数组遍历完。
- 伪代码：

```
输入: 数组 a[0..N-1], 当前考虑的元素个数 N, 目标值 key
输出: 如果找到返回 1, 否则返回 0

if N == 0 then
    return 0
end if
if a[N-1] == key then
    return 1
else
    return Recursive_Sequential_Search(a, N-1, key)
end if
```

- 实际代码：

```c
int Recursive_Sequential_Search(int a[], int N, int key) {
    if (N == 0) {
        return 0;
    }
    if (a[N - 1] == key) {
        return 1;
    }
    return Recursive_Sequential_Search(a, N - 1, key);
}
```

2、程序实现

1. **主要数据结构**
   采用有序数组a[0..N-1],元素值等于其下标(a[i]=i)来表示有序列表,目标值设置为N+1以所检索的值不在列表内，保证分析最坏情况的运行时间。
2. **主要算法**
   i. **初步思路**：对每个算法，单独使用<time.h>来测试算法性能，手动输入数据规模N和K根据ticks的取值，重新更改K的取值直到ticks>10满足要求，一下为伪代码演示。

```
输入: 整数 N, K, key
输出: 打印迭代二分搜索的平均执行时间
2. 初始化数组 a[0..N-1]
for i = 0 to N-1:
    a[i] = i
3. 记录开始时间 start = current_time()   // 获取当前时钟 tick 数
4. for i = 0 to K-1:
    调用 Iterative_Binary_Search(a, N, key)   // 在数组 a 中搜索 key
5. 记录结束时间 stop = current_time()
6. 计算总 ticks 数: total_ticks = stop - start
7. 计算总时间（秒）: total_time = total_ticks / CLOCKS_PER_SEC
8. 计算单次平均时间: average_time = total_time / K
9. 打印输出:
"Iterative Binary Search: total_ticks ticks, Total Time: total_time seconds, Duration:
```

ii. **反思**：1.K的取值需要不停修改，效率低；2.四个函数分别进行上述操作，有大量重复代码，代码不整洁

iii. **改进**：1.通过算法，以K=10开始，如果ticks不符合要求则不断递增，直到找到合适的K值，实现自动获取K取值
2.将上述流程封装为函数Test_Algoritms,使测试程序更加简洁。

iv. **最终算法实现**

```
函数 Test_Algorithms(a, N, key, search_func)
输入: 整型数组 a[0..N-1], 数组长度 N, 要查找的值 key, 搜索函数 search_func
输出: 打印一行包含 N, K, ticks, total_time, duration 的统计信息

13. 初始化 K = 10
14. 无限循环:
    a. 记录开始时间 start = current_time()
    b. 重复 K 次:
        调用 search_func(a, N, key)
    c. 记录结束时间 stop = current_time()
    d. 计算 ticks = stop - start
    e. 计算总时间 total_time = ticks / CLOCKS_PER_SEC
    f. 计算平均时间 duration = total_time / K
    g. 如果 ticks > 10 或者 K > 1000000:
        跳出循环
    否则:
        K = K + 2
15. 打印输出格式:
"%6d %10d %10ld %12.6f %12.8f \n" 对应 N, K, ticks, total_time, duration
```

```c
void Test_Algorithms(int a[], int N, int key, int (*search_func)(int[], int, int)) {
    int K = 10;
    while (1) {
        start = clock();
        for (int i = 0; i < K; i++) {
            search_func(a, N, key);
        }
        stop = clock();
        ticks = stop - start;
        total_time = (double)ticks / CLOCKS_PER_SEC;
        duration = total_time / K;
        if (ticks > 10 || K > 1000000) {
            break;
        }
        K += 2;
    }
    printf("%6d %10d %10ld %12.6f %12.8f \n", N, K, (long)ticks, total_time, duration)
}
```

v. **BUG**：该Test_Algoritms需要传入函数指针，但是不同的查找算法具有不同的参数形式，（如递归二分查找需要共计四个参数，其他函数只需三个）直接调用会报错。
vi. **BUG的修复**：为了统一函数的指针类型，我为递归二分查找写了一个包装函数Recursive_Binary_Search_Wrapper。该函数将外部传入的数组、长度和关键字转换为递归函数所需的初始边界参数 0 和 N-1，从而使接口变得一致。

```
int Recursive_Binary_Search_Wrapper(int a[], int N, int key) {
    return Recursive_Binary_Search(a, 0, N - 1, key);
}
```

主程序实现

```
int main() {
    int N;
    scanf("%d", &N); /*Read the size of the array*/
    int key = N + 1; /*Use a value not in the array*/
    int a[N];
    for (int i = 0; i < N; i++) {
        a[i] = i; /*Fill the array with sequential values*/
    }
    printf("Algorithm                   %6s %10s %10s %12s %12s
     \n", "N", "K", "Ticks",
           "Total Time", "Duration");
    // Measure the time taken by Iterative Binary Search
    printf("Iterative Binary Search     ");
    Test_Algorithms(a, N, key, Iterative_Binary_Search);

    // Measure the time taken by Recursive Binary Search
    printf("Recursive Binary Search     ");
    Test_Algorithms(a, N, key, Recursive_Binary_Search_Wrapper);

    // Measure the time taken by Iterative Sequential Search
    printf("Iterative Sequential Search ");
    Test_Algorithms(a, N, key, Iterative_Sequential_Search);

    // Measure the time taken by Recursive Sequential Search
    printf("Recursive Sequential Search ");
    Test_Algorithms(a, N, key, Recursive_Sequential_Search);

    return 0;
}
```

Chapter 3：测试结果

1、输出样列

```
10000
Algorithm                        N          K      Ticks   Total Time     Duration
Iterative Binary Search      10000         90         11     0.000011   0.00000012
Recursive Binary Search      10000         74         25     0.000025   0.00000034
Iterative Sequential Search  10000         10        236     0.000236   0.00002360
Recursive Sequential Search  10000         10       2142     0.002142   0.00021420
```

2、表格

表格1：N=100~2000

	N	**100**	**500**	**1000**	**2000**
Binary Search (iterative version)	Iterations (K)	234	98	112	98
	Ticks	25	22	11	11
	Total Time (sec)	0.000025	0.000022	0.000011	0.000011
	Duration (sec)	0.00000011	0.00000022	0.00000010	0.00000011
Binary Search (recursive version)	Iterations (K)	62	112	92	86
	Ticks	11	11	11	11

	N	100	500	1000	2000
	Total Time (sec)	0.000011	0.000011	0.000011	0.000011
	Duration (sec)	0.00000018	0.00000010	0.00000012	0.00000013
Sequential Search (iterative version)	Iterations (K)	54	10	10	10
	Ticks	11	13	29	56
	Total Time (sec)	0.000011	0.000013	0.000029	0.000056
	Duration (sec)	0.00000020	0.00000130	0.00000290	0.00000560
Sequential Search (recursive version)	Iterations (K)	10	10	10	10
	Ticks	24	86	238	489
	Total Time (sec)	0.000024	0.000086	0.000238	0.000489
	Duration (sec)	0.00000240	0.00000860	0.00002380	0.00004890

表格2：N=4000~10000

	N	4000	6000	8000	10000
Binary Search (iterative version)	Iterations (K)	72	92	90	86
	Ticks	11	11	11	11
	Total Time (sec)	0.000011	0.000011	0.000011	0.000011
	Duration (sec)	0.00000015	0.00000012	0.00000012	0.00000013
Binary Search (recursive version)	Iterations (K)	84	80	106	74
	Ticks	11	11	11	11
	Total Time (sec)	0.000011	0.000011	0.000011	0.000011
	Duration (sec)	0.00000013	0.00000014	0.00000010	0.00000015
Sequential Search (iterative version)	Iterations (K)	10	10	10	10
	Ticks	101	152	188	268
	Total	0.000101	0.000152	0.000188	0.000268

	N	4000	6000	8000	10000
	Time (sec)				
	Duration (sec)	0.00001010	0.00001520	0.00001880	0.00002680
Sequential Search (recursive version)	Iterations (K)	10	10	10	10
	Ticks	919	961	1538	1730
	Total Time (sec)	0.000919	0.000961	0.001538	0.001730
	Duration (sec)	0.00009190	0.00009610	0.00015380	0.00017300

3、性能曲线图

函数拟合图

4、图像分析

1. 两种顺序查找算法在N较大时接近线性递增，但N较小时偏离拟合函数过多
2. 递归版本的顺序查找比迭代版本的斜率基本一致，但是花费时间大于迭代版本，证明了递归版本因为调用函数有开销，实际运行时间常数因子大。
3. 两种二分查找算法，基本符合O(log N)复杂度，但在N较小的情况下仍然有波动。

5、误差分析

1. **当N小于2000时数据拟合效果不佳**原因可能有以下几点：
    i. **K取值不当**：clock() ticks是整数，最小单位为 1/CLOCKS_PER_SEC 秒。当单次运行时间极短时，需要靠微小量放大法重复 K 次来累积足够的 ticks。但我到数据中多数

小 N 的 ticks 仅为 11~13，刚好超过题目要求的 10 个 tick。导致计算出的总时间有相对误差。造成小幅偏离理论线性。

ii. **系统环境噪声**：查询资料得知，系统的后台进程，CPU频率调整，中断处理等会引入随机延迟，N较小时运行时间短，噪声的影响大，导致个别数据点波动；此外，查询资料得知，递归调用本身的不确定性（如栈访问延迟）可能也放大了噪声。使得误差更大。

iii. **重复测量次数过少**，使得随机误差被放大

2. **误差大小**：通过**相对误差分析**（即计算实测时间与理论 线性拟合的期望值的相对偏差）实际计算得知大多偏差都在正负10%以内,与题目要求的10%精度相符合

6、实验环境

1. **操作系统**：
   - 发行版：Ubuntu 24.04.4 LTS (Noble Numbat)
   - 内核版本：6.17.0-19-generic
2. **编译器与工具链**：
   - 编译器：gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
   - 调试器：GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
   - 编译选项：-g，未开启优化，保证获取递归调用的原始运行时间。

Chapter 4：分析与评论

复杂度分析

1、时间复杂度

1. **二分搜索（迭代和递归）**

- Worst Case: 目标不在数组，需要将搜索区间缩减到0
- 推导：初始区间长度为N，每次比较后，区间长度减半，变为N/2，将搜索区间减为0共需要[$\log_2 N$]+1次对比
- **时间复杂度: O(log N)**

2. **顺序搜索（迭代和递归）**

- Worst Case: 目标值不在数组，或是数组第一/最后一个值，需要完整遍历数组，比较N个元素
- 推导：迭代版本通过循环遍历，每次进行一次比较，遍历数组共需要N次比较；递归版本每次递归调用减少一个元素，共递归N次，每次递归进行一次比较，共需要N次遍历
- **时间复杂度: O(N)**

2、空间复杂度

1. **迭代版本**
   由于使用固定的额外空间，**空间复杂度为O(1)**
2. **递归版本**
   每一层递归占用常数空间，递归深度为$\log_2 N+1$,综上，空间**复杂度为O(log N)**

3、递归对于运行时间的影响:

- 由于递归版本因为调用函数有开销，实际运行时间常数因子大。

4、补充反思

1. 虽然递归和迭代版本在时间复杂度的分析中，时间复杂度一致，但由于递归版本**调用函数**有额外开销，其实际运行常数因子大，在数据量较小的时候会显著影响实际运行时间。
2. 有误差分析部分可知，递归调用本身的不确定性（如栈访问延迟）可能会放大了噪声。使得误差更大。
3. 该题中的两个算法的**递归版本**都是**尾递归**，因此有对应的**迭代版本**,说明为尾递归在实际运行中，性能不如对应的迭代版本

进一步改进

1. 可以将**尾递归**改成**迭代**以减小开销，或是减小误差的影响
2. 实验数据应该多次测量取平均值以减少**系统环境噪声**等随机误差的影响。
3. K值可以适当取更大值，以确保数据的精确程度。

声明

我在此声明项目“*Performance Measurement*"里的所有的内容均由我自己独立完成
I hereby declare that all the work done in this project titled "*Performance Measurement*" is of my independent effort.